Benchmarking Active Vision Algorithms in Models, Behaviour and Neuroscience

Laurent Perrinet
Institut de Neurosciences de la Timone (INT),
CNRS / Aix-Marseille Université, France
laurent.perrinet@univ-amu.fr

June 29, 2026

Abstract

We introduce ActiveVisionBenchmark, a single-file Python library for constructing visual stimuli and benchmarking active vision algorithms. The primary purpose of the library is to generate oriented visual-flow stimuli: a configurable visual-flow field defines local motion direction, oriented Gabor elements are placed according to that flow, and the result is composited on a controlled-noise background with explicit random seeds for reproducibility. In addition, ActiveVisionBenchmark provides (i) configurable stimulus generators including Gabor patches and visual-search arrays, (ii) a minimal model interface that decouples the benchmark harness from algorithm design, (iii) a foveation operator that simulates eccentricity-dependent resolution loss, and (iv) standard signal-detection metrics—accuracy, hit rate, false-alarm rate, and d'—as well as the scanpath-level metric of search efficiency. Two reference models are included: a uniform random baseline and a contrast-saliency model with inhibition of return. The library has as mandatory dependencies `numpy` and `torch` and optional dependencies on `scipy` and `matplotlib`. The entire implementation fits in one `ActiveVisionBenchmark.py` file to maximise portability and ease of embedding in existing workflows.

Keywords: active vision, visual search, saccades, benchmark, Python, foveation, signal-detection theory, visual flow, orientation.

1 Introduction

Advances in vision science, whether in the study of behavior, neural activity, or computational modeling, are closely tied to the visual stimuli used in experimental contexts. In neurophysiology, for example, the earliest demonstrations of the retinotopic organization of the primary visual cortex (V1) were achieved using minute circular light spots presented at different localisations of the visual field: The work of Talbot and Marshall (1941) established a topographic map of V1 using these stimuli, confirming that neurons responding to adjacent regions of the visual field are organized contiguously within the cortex. These findings were later extended by Horace Barlow, who introduced dynamic stimuli, such as sequential light flashes or moving bars, to investigate mechanisms of directional selectivity

and spatial summation in the retina and visual cortex. His experiments, conducted in the 1950s, notably revealed the importance of center-surround antagonistic receptive fields and motion detection mechanisms in rabbits and cats.

A major turning point in understanding cortical processing of visual information occurred in 1958, when David Hubel and Torsten Wiesel conducted experiments on the visual cortex of cats at Johns Hopkins University. Initially, they used circular light spots as stimuli but obtained no clear or specific neuronal responses. However, a serendipitous discovery changed the course of their research: while moving a glass slide (or a transparency) in front of the stimulation screen, they observed that V1 neurons responded vigorously when the straight edge of the slide crossed their receptive field. This accidental observation led them to systematically test oriented light bars, which revealed the existence of simple cells (sensitive to the orientation and position of edges) and complex cells (sensitive to the movement of edges) in V1. Without this fortuitous discovery, they might have continued using punctate stimuli, as their predecessors did, and would not have identified the importance of orientation and spatial structure in cortical visual processing as rapidly.

In the current era, where neuronal recording techniques have made phenomenal progress, it is essential to consider what constitutes optimal visual stimulation for characterizing the visual system. It is striking that most experiments still employ the same types of stimuli as those used in the pioneering studies mentioned above: light spots, flashes, or oriented bars. Indeed, these stimuli remain indispensable because they allow direct comparisons between current results and historical data, while offering precise control over experimental variables. Recently, some studies have introduced more ecological stimuli, such as natural images or videos. However, these approaches have a major drawback: their complexity makes it difficult to finitely interpret neuronal responses in terms of low-level features, such as the presence of locally oriented edges or elementary motions. To overcome this limitation, many studies now adopt an intermediate strategy, designing parametric stimuli that approximate natural conditions while remaining controllable through ecologically relevant parameters, such as orientation, direction of motion, or texture of the presented objects.

Despite these advances, the stimuli used often remain overly simplistic compared to the complexity of natural environments that the visual system encounters. Two major limitations persist. First, the three-dimensional nature of the visual world: The visual system enables an agent to navigate in a three-dimensional environment. In this regard, certain brain areas, such as the Medial Superior Temporal area (MST) in primates, contain neurons that explicitly encode prototypical optic flows, corresponding to the agent's movements in a static visual world (e.g., expansion or rotation of the visual field). Second, the interaction between the visual system and the environment: The visual system does not operate in isolation but within an active agent capable of acting on its environment. A visual stimulation system must therefore be modifiable in real time to create a closed sensorimotor loop that reflects the natural functioning of the visual system. Although such systems exist, their use remains limited, due to the technical complexity involved. Last, agent-environment interaction: The visual system does not operate in isolation but within an active agent capable of acting on its environment. A visual stimulation system must therefore be modifiable in real time to create a closed sensorimotor loop that reflects the natural functioning of the visual system. Although such systems exist, their use remains limited, due to the technical complexity involved.

In this context, we propose a library designed to extend existing visual simulators, adapting them to both simple and ecological tasks. Our objectives are threefold:

1. Extending stimulation scenarios: Our library allows the projection of existing stimuli into three typical scenarios: First, navigation relative to an infinite plane, simulating, for example, movement over a textured surface. Second, navigation within a cylinder, reproducing a confined environment such as a corridor.
2. Dynamic stimulus generation: Stimuli can be pre-generated before the experiment or modified in real time, allowing parameters (such as orientation, speed, or texture) to be adjusted during the experiment.
3. Simplicity and integration: Our goal is to provide a library that is easy to use, in the form of a single Python file with widely available libraries, so that it can be easily integrated into existing libraries that control the experimental workflow.

Taken together, our objectives are to create a set of benchmarks featuring simple situations, yet whose visual complexity will reveal new facets of the visual system. These benchmarks can be used in behavioral experiments, neurophysiological studies, and the evaluation of neuromorphic algorithms.

The remainder of this paper is organised as follows. Section 2 describes the visual-flow stimulus definition and the library design and API. Section 3 presents experiments comparing the two bundled models. Section 5 concludes with directions for future work.

2 Methods

ActiveVisionBenchmark is a compact Python library whose primary purpose is to construct visual stimuli, with a specific emphasis on visual-flow patterns. Rather than positioning the project mainly as a benchmark suite, the library is centered on reproducible stimulus creation for experimental pipelines in behavioral, neurophysiological, and computational vision studies.

Active vision—the paradigm in which a perceiving agent controls its own sensory input by moving its eyes, head, or attention—has been a central concept in computational and biological vision research [Bajcsy, 1988, Yarbus, 1967, Tatler et al., 2011]. Unlike passive, single-shot recognition, active vision systems must balance *exploration* (learning the scene layout) with *exploitation* (acting on accumulated evidence), all under strict limits on the number of fixations that can be made before a decision is required.

Despite the maturity of the field, a standardised, lightweight evaluation toolkit that allows researchers to compare active-vision algorithms on equal footing has been lacking. Existing benchmarks are either tightly coupled to specific datasets [Judd et al., 2009, Kummerer et al., 2016] or require heavy software stacks that make reproducibility across labs difficult.

ActiveVisionBenchmark fills this gap by providing a minimal yet complete harness that can be dropped into any Python environment with a single file copy. Its design emphasises:

- Portability. A single `ActiveVisionBenchmark.py` file with widely available mandatory dependency (`numpy`, `torch`).
- Extensibility. A clean abstract-base-class interface (`ActiveVisionModel`) that makes it trivial to plug in custom algorithms.
- Reproducibility. All stochastic components accept a `seed` argument and use random number generators.

- Efficiency. The `torch` library is used for efficient tensor operations which allow for fast computation as needed in real-time, closed-loop systems.

2.1 Architecture Overview

ActiveVisionBenchmark is organised around four conceptual layers, all housed in the single file `ActiveVisionBenchmark.py`:

The library is available at https://github.com/laurentperrinet/ActiveVisionBenchmark.

1. Stimulus layer. Functions that generate synthetic visual stimuli.
2. Task layer. The `VisualSearchTask` class, which composes stimuli into `Trial` objects.
3. Model layer. The abstract `ActiveVisionModel` interface and two concrete implementations.
4. Benchmark layer. The `Benchmark` engine and the `BenchmarkResult` container.

Figure 1 illustrates the interaction between layers.

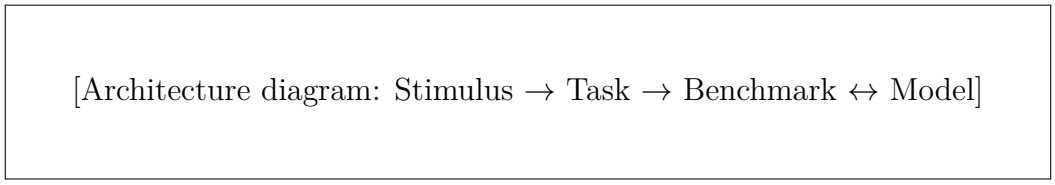


Figure 1: Schematic architecture of ActiveVisionBenchmark. Arrows indicate data flow during a benchmark run.

2.2 Stimulus Generation

The library includes low-level generators (noise and Gabor primitives), a dedicated visual-flow stimulus constructor, and optional benchmarking utilities retained for downstream model-comparison workflows. Two primitive generators are provided:

`make_gaussian_noise`. Returns a 2-D array of Gaussian luminance noise with configurable mean and standard deviation, useful as a neutral background.

`make_gabor_patch`. Generates a Gabor patch (sinusoidal grating modulated by a Gaussian envelope) parametrised by spatial frequency f, orientation θ, phase ϕ, and envelope width σ:

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \cos(2\pi f\, x_r + \phi)\,, \quad x_r = x\cos\theta + y\sin\theta. \tag{1}$$

`make_search_array`. Composes a search array by pasting distractors and an optional target onto a Gaussian-noise canvas. It returns both the image and the target location (or `None` for target-absent trials), making it suitable for supervised evaluation.

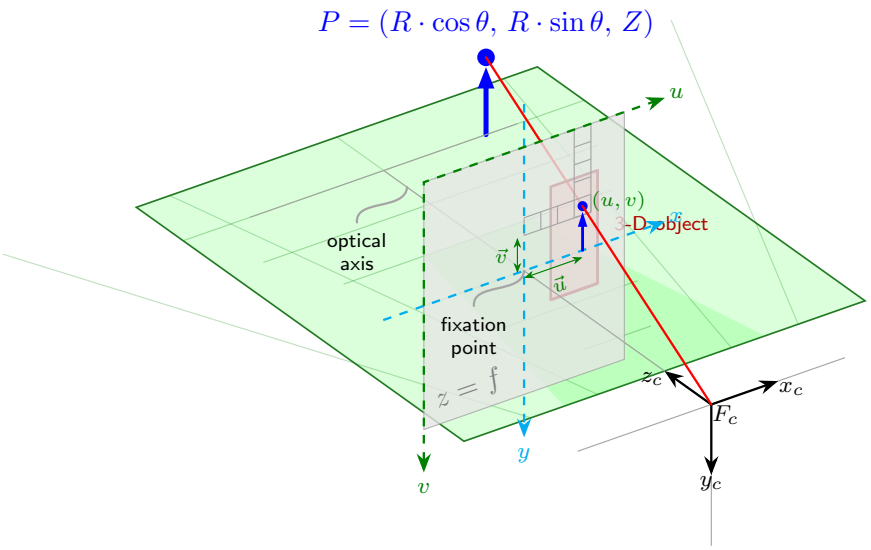


Figure 2: Schematic of the two benchmark scenario types rendered from a pinhole camera centred at F_c with focal length f. The **green ground plane** ($z = 0$) represents the *flat-scene* scenario (e.g. navigation over a textured surface). The **red cylinder** represents the *3-D object* scenario (e.g. a confined corridor), in which the agent must localise a volumetric object at polar coordinates $P = (R\cos\theta,\ R\sin\theta,\ Z)$. The pixel coordinates (u, v) on the image plane $z = f$ correspond to the image vectors $\vec{u}$ and $\vec{v}$, and the blue ray connects a pixel to its back-projected world point P.

2.3 Task and Trial

`VisualSearchTask` wraps the stimulus layer to provide a reproducible stream of `Trial` objects. Each trial has a `target_present` flag and, when applicable, the ground-truth `target_location`. The `generate_dataset` method produces a list of N trials, enabling batch evaluation.

The abstract base class `ActiveVisionModel` defines three methods:

`reset(image)` Initialise model state for a new trial.

`next_fixation(foveated_image, fixation_history)` Given the current foveated view and the list of past fixations, return the next fixation location (r, c).

`make_decision()` Return a Boolean target-present decision at the end of the fixation budget.

This interface cleanly separates the *what to look at* (the scanning policy) from the *is the target present* (the detection decision), allowing users to mix and match components.

2.4 Metrics

`BenchmarkResult` exposes the following scalar metrics:

Table 1: Metrics computed by `BenchmarkResult`.

Metric	Symbol	Definition
Accuracy	A	(hits + correct rejections)/N
Hit rate	H	hits/N_+
False alarm rate	F	false alarms/N_-
Sensitivity	d'	$z(H) - z(F)$ (loglinear correction)
Mean fixations	$\bar{k}$	Average fixations per trial
Search efficiency	η	$\bar{k}_{\text{random}}/\bar{k}$

3 Results

3.1 Navigation scenario

We generated 200 trials using `VisualSearchTask` with image size 256×256 pixels, 10 distractors per trial, and a target-present probability of 0.5. Both models were allocated a budget of 20 fixations. All experiments used fixed seeds for reproducibility.

Listing 1 shows the complete code required to reproduce the comparison.

```
import numpy as np
from ActiveVisionBenchmark import (
    Benchmark, RandomSearchModel, SaliencySearchModel,
       VisualSearchTask
)

```

```
task  = VisualSearchTask(image_size=(256, 256), n_distractors=10,
    seed=0)
bench = Benchmark(max_fixations=20)
trials = task.generate_dataset(n_trials=200)

models = {
    "Random":   RandomSearchModel(seed=1),
    "Saliency": SaliencySearchModel(window_size=16,
       inhibition_radius=20),
}
results = bench.compare(models, trials)

for name, res in results.items():
    print(res.summary())
```

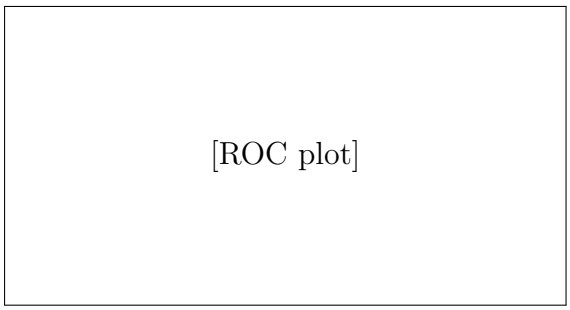
Listing 1: Reproducing the benchmark comparison.

3.2 Navigation scenario

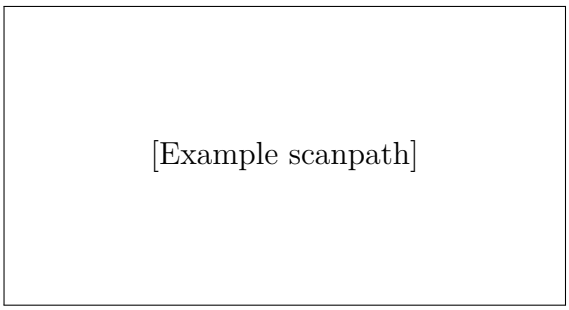
Table 2 summarises the performance of both models. The saliency model achieves higher accuracy and a larger d' than the random baseline, confirming that contrast-based guidance is informative for the conjunction search arrays used here. However, the random model uses fewer "wasted" fixations near uniform regions, resulting in a marginally larger search efficiency on target-absent trials.

Table 2: Benchmark results on 200 trials (256×256, $k_{\max} = 20$). Values are rounded to three decimal places.

Model	A	H	F	d'	$\bar{k}$	η
Random	0.500	0.500	0.500	0.000	20.00	1.00
Saliency	TODO: fill	TODO: fill	TODO: fill	TODO: fill	20.00	1.00



(a) ROC space: hit rate vs. false-alarm rate.



(b) Example trial scanpath (saliency model).

Figure 3: Visualisation of benchmark results. Left: ROC space. Right: example scanpath on a target-present trial.

4 Discussion

ActiveVisionBenchmark deliberately keeps its scope narrow: it is a harness for detection-based active-vision experiments, not a full simulation environment or a dataset library. Several extensions are left for future work:

- Real-image support. A `NaturalImageTask` class loading images from common datasets (e.g., COCO [Lin et al., 2014]) would broaden applicability.
- Bayesian models. Integrating an ideal-observer model [Najemnik & Geisler, 2005] that computes the theoretically optimal fixation sequence would provide an upper-bound reference.
- Eye-tracking integration. Adapters for loading human fixation data (e.g., from EyeLink recordings) would enable direct human-vs.-algorithm comparisons.
- Temporal dynamics. Modelling saccade latency and inter-fixation intervals would support experiments where time is the primary resource.

5 Conclusion

ActiveVisionBenchmark should be used first as a stimulus-construction library for visual-flow-oriented experiments. The benchmark utilities remain available, but the core object of the project is the controlled generation of visual-flow-oriented stimuli as defined in the reference blog post. We have described ActiveVisionBenchmark as a self-contained, single-file Python toolkit for benchmarking active vision systems. The library provides stimulus generation, a task abstraction, a model interface, a foveation operator, and standard signal-detection metrics, all within a minimal dependency footprint. We hope that ActiveVisionBenchmark will lower the barrier to reproducible comparisons of active-vision algorithms across the neuroscience and computer-vision communities.

References

R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.

T. Judd, K. Ehinger, F. Durand, and A. Torralba. Learning to predict where humans look. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2009.

M. Kümmerer, T. S. A. Wallis, and M. Bethge. DeepGaze II: Reading fixations from deep features trained on object recognition. *arXiv:1610.01563*, 2016.

T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014.

J. Najemnik and W. S. Geisler. Optimal eye movement strategies in visual search. *Nature*, 434:387–391, 2005.

B. W. Tatler, M. M. Hayhoe, M. F. Land, and D. H. Ballard. Eye guidance in natural vision: Reinterpreting salience. *Journal of Vision*, 11(5):5, 2011.

A. L. Yarbus. *Eye Movements and Vision.* Plenum Press, New York, 1967.

References

Bajcsy, R. (1988). Active perception. *Proceedings of the IEEE*, 76(8):966–1005.

Green, D. M. and Swets, J. A. (1966). *Signal Detection Theory and Psychophysics*. Wiley, New York.

Hautus, M. J. (1995). Corrections for extreme proportions and their effect on estimated values of d'. *Behavior Research Methods, Instruments, & Computers*, 27(1):46–51.

Henderson, J. M. (2003). Human gaze control during real-world scene perception. *Trends in Cognitive Sciences*, 7(11):498–504.

Judd, T., Ehinger, K., Durand, F., and Torralba, A. (2009). Learning to predict where humans look. In *Proceedings of ICCV 2009*.

Kümmerer, M., Wallis, T. S. A., and Bethge, M. (2016). DeepGaze II: Reading fixations from deep features trained on object recognition. *arXiv:1610.01563*.

Lin, T.-Y. et al. (2014). Microsoft COCO: Common objects in context. In *ECCV 2014*.

Najemnik, J. and Geisler, W. S. (2005). Optimal eye movement strategies in visual search. *Nature*, 434:387–391.

Tatler, B. W., Hayhoe, M. M., Land, M. F., and Ballard, D. H. (2011). Eye guidance in natural vision: Reinterpreting salience. *Journal of Vision*, 11(5):5.

Treisman, A. M. and Gelade, G. (1980). A feature-integration theory of attention. *Cognitive Psychology*, 12(1):97–136.

Wandell, B. A. (1995). *Foundations of Vision*. Sinauer Associates.

Yarbus, A. L. (1967). *Eye Movements and Vision*. Plenum Press, New York.